09. Prototype Guide

AdmitAssist | MBA ZG583 Management of AI Products | GOLLAPUDI SIVA NAGA JNANA TEJA | 2025MB26205 | GitHub: jnanateja009

1. What the prototype is

A working web app (plain HTML and JavaScript, no install, no framework) that runs the full V1 decision loop: paste an inquiry, the rules and the AI read it, it is routed and prioritised with reasons, a draft is written and checked, a person approves or rewrites it, opens WhatsApp with the text, and every action is logged with the pilot metrics.

The same engine file (`prototype/lib/engine.js`) runs in the browser and in the evaluation script, so the numbers in 05 come from the exact logic on screen.

2. Run it

Way	Steps	Needs
Local web server (recommended)	`cd prototype` then `python -m http.server 8000`, open http://localhost:8000	Python 3
Double-click	Run `node tools/build_bundle.js` once inside `prototype/`, then open `index.html`	Node 18+ once
GitHub Pages	Repository Settings > Pages > Deploy from branch `main`, folder `/ (root)`; open `https://maip2026july.github.io/<repo-name>/prototype/`	Public repository

Browsers: Chrome or Edge, current and previous major version (NFR-07).

2.1 Recorded demo (no key)

Click **Load sample messages**. The 48 written samples are read using recorded AI responses (05, section 5). Everything after the AI step (quote checks, priority, reply-by time, draft checks, approval, log) runs live in the browser.

A message you paste yourself in this mode has no recorded response, so you will see the fallback path: "The AI could not read this message. Please read it yourself", reply-by 2 working hours, top of the queue. That is the designed behaviour when the AI service is down.

2.2 Live AI

1. Open **AI connection**, choose **Live AI**.
2. Service address: `https://api.openai.com/v1` (or any OpenAI-compatible address, for example Gemini's OpenAI-compatible endpoint).
3. Model: `gpt-6.1-terra` (default from settings).
4. Paste an API key. It stays in this browser tab's session storage and is cleared when the tab closes. In the pilot build the key moves to the server (04, deployment diagram).
5. **Test connection**, then **+ New inquiry**.

Monthly AI cap: when this browser has spent Rs 500 in a month (`llm.monthly_ai_cap_inr`), AI calls stop and messages go to "Read it yourself". In the pilot build the cap is per school, on the server.

Other buttons on a card: **Open e-mail** (when the message has an e-mail address), a number picker when a message has two mobiles, and **Report a problem with this reply** on done inquiries. Closing a staff-only or not-admission card asks for a one-line note.

3. Three-minute demo script (for the presentation)

#	Do	Point to make
1	Load sample messages. Show the queue sorted by reply-by time.	Serious parents first; overdue in red
2	Open INQ-004 (relocating, Class 6). Show "Why this priority" with points and quotes, and the Sure tags.	The AI reports, the rules decide, every value has proof
3	Edit the draft: type "your seat is confirmed". The check line turns red. Remove it. Approve.	Checks run on every keystroke; the tool cannot send
4	Open INQ-043 (speech delay, wants to visit Saturday). Purple Staff only card, no draft.	Rules alone would have drafted a cheerful reply; the AI caught it
5	Open INQ-026 (prompt injection). Review note; draft gives the approved fee range, no seat promise.	Message is treated as data
6	Open School settings, mark Class 9 seats "closed", save.	New class, new fee, new rule: settings, not code
7	Open Activity log.	The pilot metrics are measured from day one

4. Files

Path	What it is
`index.html`, `styles.css`, `app.js`	The inbox app
`lib/engine.js`	All rules, prompts and the pipeline (shared with the evaluation)
`lib/llm.js`	AI connectors: OpenAI-compatible live calls, and replay of recorded responses
`config/school_profile.json`	Sample school settings (fictional "Greenfield Public School")
`config/baseline_rules.json`	The rules-only reader used as the baseline
`data/sample_inquiries.json`	48 written sample messages with provisional labels
`data/recorded_ai_responses.json`	Recorded AI answers to the exact `read_v1` and `draft_v1` prompts
`eval/run_eval.js`	Evaluation: `--mode baseline`, `--mode replay`, `--mode live`, `--compare`, `--export read
`eval/unit_tests.js`	51 unit and red-team tests
`eval/token_profile.js`	Average tokens per AI step (feeds `business/unit_economics.py`)
`eval/results/`	All result files quoted in 05
`tools/build_bundle.js`	Packs settings and data into `bundle.js` so `index.html` opens from disk

5. Reproduce every number

```
cd prototype
node eval/unit_tests.js                # 51/51 pass
node eval/run_eval.js --mode baseline   # rules only
node eval/run_eval.js --mode replay     # recorded AI run
node eval/run_eval.js --compare         # eval/results/comparison.md
node eval/token_profile.js              # tokens per step
cd .. && python business/unit_economics.py
```

Live run (Gate A): `OPENAI_API_KEY=... node eval/run_eval.js --mode live` writes `eval/results/live_summary.md` with real token counts and timings.

6. What the prototype proves and what it does not

Proves	Does not prove
The full loop works end to end, in the browser and in the evaluation	That it works on real parents' messages (D2, Phase 0)
The rules (masking, keyword net, class patterns, quote check, priority points, reply-by time with holidays, draft checks, locked safety settings) behave as specified: 51 of 51 tests	That gpt-6.1-terra through the API behaves like the stand-in model used for the recorded run
A language model can catch indirect and non-English sensitive messages that rules miss (10/10 vs 4/10 on samples)	That sensitive recall is above 95% (needs 75 sensitive messages with no miss, Gate A)
Everything school-specific is in settings	That staff will use it carefully in a real season (pilot)
AI cost per inquiry: Rs 0.32 measured with estimated tokens (Rs 0.36 planning figure with retries)	Exact cost and latency (live run)

7. Known limitations

Limitation	Plan
Data lives in one browser (localStorage); two staff cannot share a queue	Server and database in Phase 1
API key in the browser tab (live mode)	Moves to the server proxy in Phase 1
No login	One PIN login per person in Phase 1 (03b section 5)
No threading of repeat messages from the same parent	Phase 1 (03b section 6)
English replies only	V2
No WhatsApp connection; paste-in only	V2
"Write again" in recorded mode has only one recorded draft per message	Works in live mode
Testing so far: 51 unit and red-team tests, both evaluation runs, and a scripted run of every screen on all 48 sample messages	Full browser test on Chrome and Edge, current and previous version, in Phase 1 (NFR-07)